ECE60827 CUDA Assignment 2
Jerry Liu, 2/25/2023

Median Filtering:

Overview:

For the median filter algorithm, the overarching strategy for parallelization is to divide the output image such that each thread computes the output of one channel per-pixel. Each index in the x and y dimension maps to the corresponding output pixel, with the z dimension accounting for the channel within each pixel:

$$\{x, y, z\} \sim \{ [0\text{–}width), [0\text{-}height), [0\text{-}channels) \}$$

In the first step of the algorithm, the threads in each thread block coordinate to load a small subset/tile of the global image into the shared memory. During this process, it is common for each thread to load more than one element/byte, so one key optimization that was made was to have each thread load in strides of the thread block size. That way, a majority of the accesses will be sequential for each warp allowing for more coalesced accesses:

Example optimized load pattern:
Thread 0: input[0], input[16], input[32] …
Thread 1: input[1], input[17], input[33] …
…

The unoptimized method, where each thread loads its elements sequentially, was also tested and is detailed in the performance charts:

Example unoptimized load pattern:
Thread 0: input[0], input[1], input[2] …
Thread 1: input[3], input[4], input[5] …
…

After loading the local tile, the threads load their windows from the local tile into a private register array to sort the values before storing them into the output.

In terms of optimization of the current approach, one of the largest issues is the unoptimized stores to the output array. While the current implementation is better than the first implementation (z = 1, serialized channel filtering), the fact that channels are assigned to the z dimension means that stores in each warp are done in strides of three instead of one (reducing coalescing). The a more optimized solution would be to increase the size of the x dimension such that every thread in the x dimension maps to one channel instead of one pixel, but unfortunately this method was not able to be

Total: 100/100

median filter: 50/50
CPU implementation: 10
GPU kernel optimizations: 10
profiling results: 10
median filtering functionality: 20

max pooling: 50/50
CPU implementation: 10
GPU kernel optimizations: 20
test cases: 10
profiling results: 10

implemented in time. There is also an issue where the filter size is fixed to a maximum of 64 elements because the size of the register array (window) must be known at compile-time. A workaround was tested where the window was allocated dynamically in shared memory, but this solution was only viable for smaller windows (<= 4). For larger windows, the additional shared memory required, along with added bank conflicts/memory latency, made this approach undesirable.

Test Results:



Figure 1: CPU Implementation - Filter Size: 4 x 4



Figure 2: GPU Implementation - Filter Size 4 x 4

ECE60827 CUDA Assignment 2
Jerry Liu, 2/25/2023

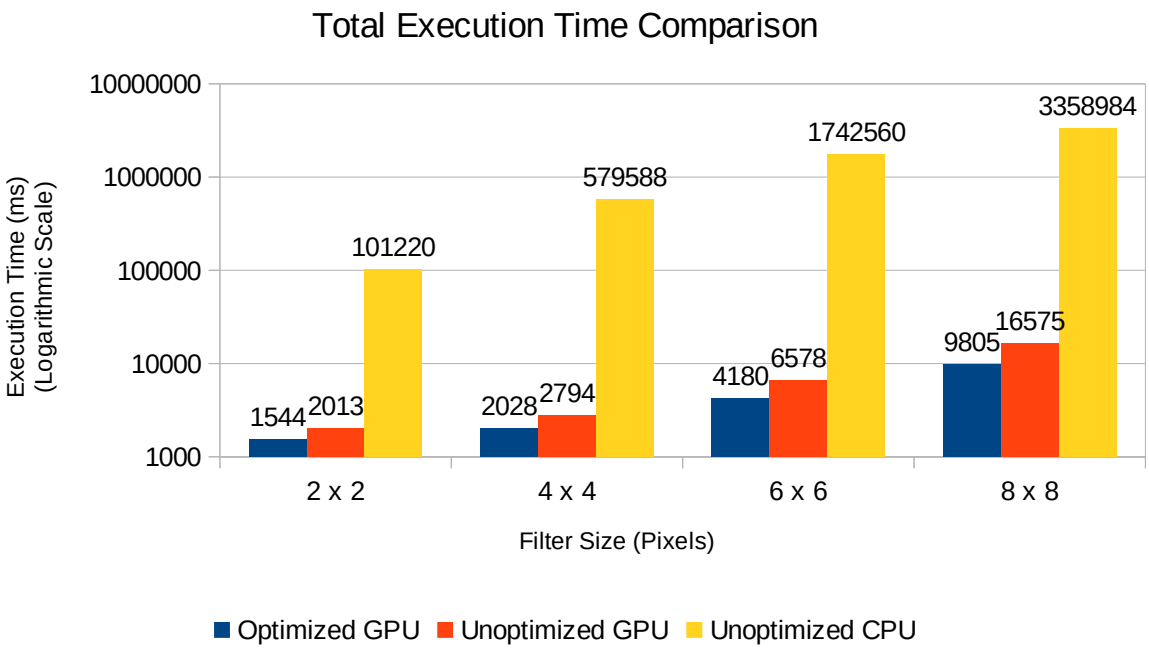


Figure 3: All execution times were measured using std:chrono at the call of the actual algorithm (i.e. medianFilter_gpu). Image used was lena512color.

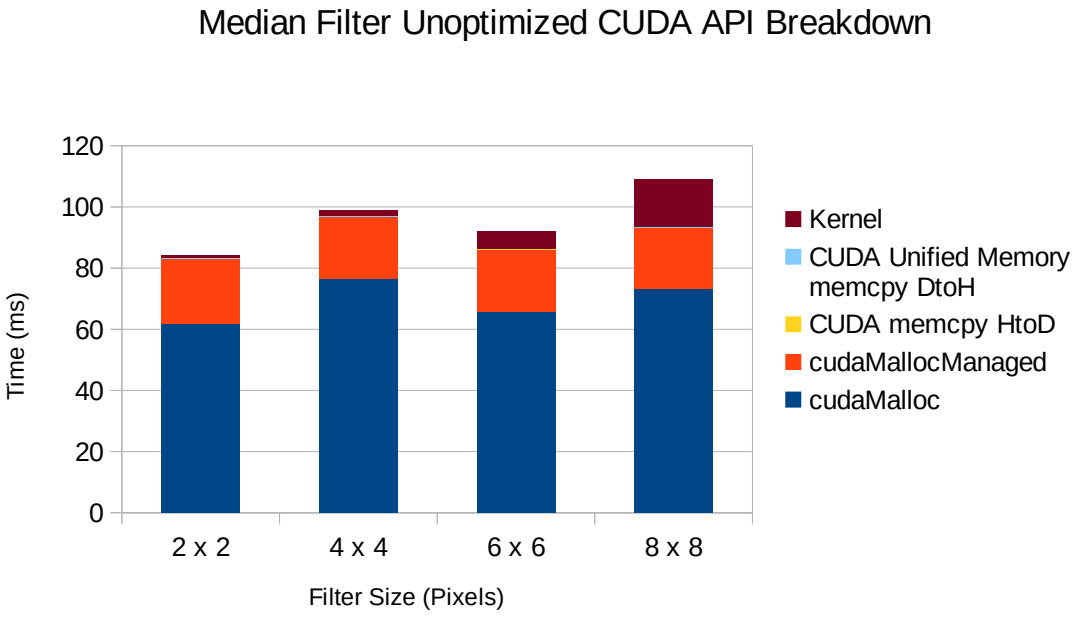


Figure 4: Breakdown of unoptimized median filter API calls using command "nsys nvprof ./lab2" Non-relevant API calls are excluded (i.e. cudaLibraryLoadData) for clarity. Image used was lena512color.

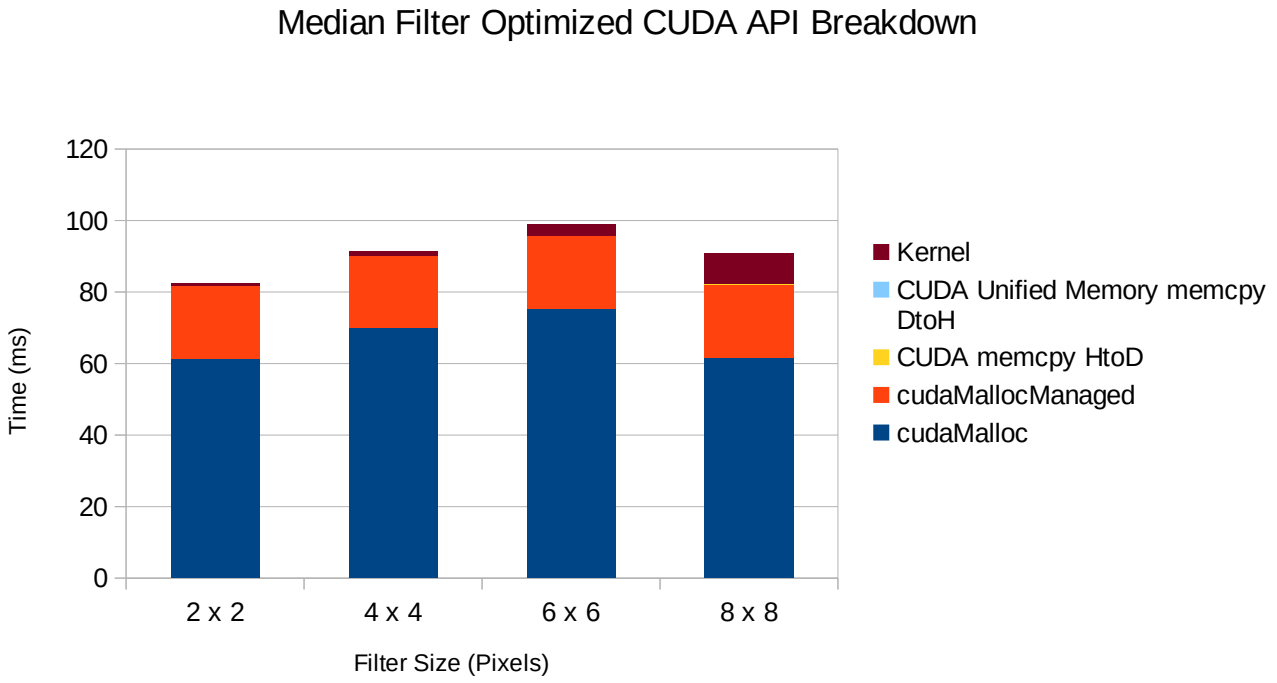


Figure 5: Breakdown of optimized median filter API calls using command "nsys nvprof ./lab2" Non-relevant API calls are excluded (i.e. cudaLibraryLoadData) for clarity. Image used was lena512color.

Observations:

Overall, the GPU implementations are much quicker than the CPU implementation (especially for larger filter sizes), while providing similar filtering results. The tests also show that there is a measurable speedup using the optimized GPU kernel across all tests. Finally, the workload appears to be “compute” bound as memory API calls are relatively constant (varies with image size), while the kernel time increases with larger filter sizes. This is also supported by analysis from nsight-compute (not shown), which reports that the kernel appears to be compute bound.

Pooling:

Overview:

The parallelization strategy for pooling is very similar to the median filter strategy. Once again, each thread in the x and y dimensions correspond to one element in the output matrix, but this time the z dimension is unused:

$$\{x, y, z\} \sim \{ [0\text{–width}), [0\text{-height}), [0] \}$$

The channels in this algorithm are handled by creating separate streams/launching multiple kernels, and, because the channels for pooling are separated by entire matrices (instead of interleaved), parallel execution of the kernels should be possible. Similar to the median filtering algorithm, the threads collectively load a subset of the input matrix into shared memory, with the stride of each thread optimized to allow for more coalesced accesses (see median filter section for details). Note that an unoptimized loading scheme was not developed for this algorithm, so performance comparisons in the testing section will be made against the CPU version only. Finally, each thread performs pooling for its output by sequentially loading the elements of the pool from shared memory, and computing as needed. Note that only one element from the pool is processed in each step, so the use of a private register array (as in median filter) is not required. Stores to the output array are also coalesced nicely since each thread outputs one value only, which allows them to be grouped sequentially into one warp with strides of one element.

Overall, the optimization of the pooling algorithm is better than the median filtering algorithm because the development of the pooling algorithm focused on efficiency from the beginning. Global accesses are better coalesced, and the overall index calculation of the algorithm is simpler. One further optimization that could be possible is to parallelize the pooling operation. This would greatly increase the efficiency of pooling (especially for larger pool sizes) by removing sequential accesses and comparisons. That said, additional threads, synchronization, and communication would be required so overall speedup is uncertain.

ECE60827 CUDA Assignment 2
Jerry Liu, 2/25/2023

Test Results:

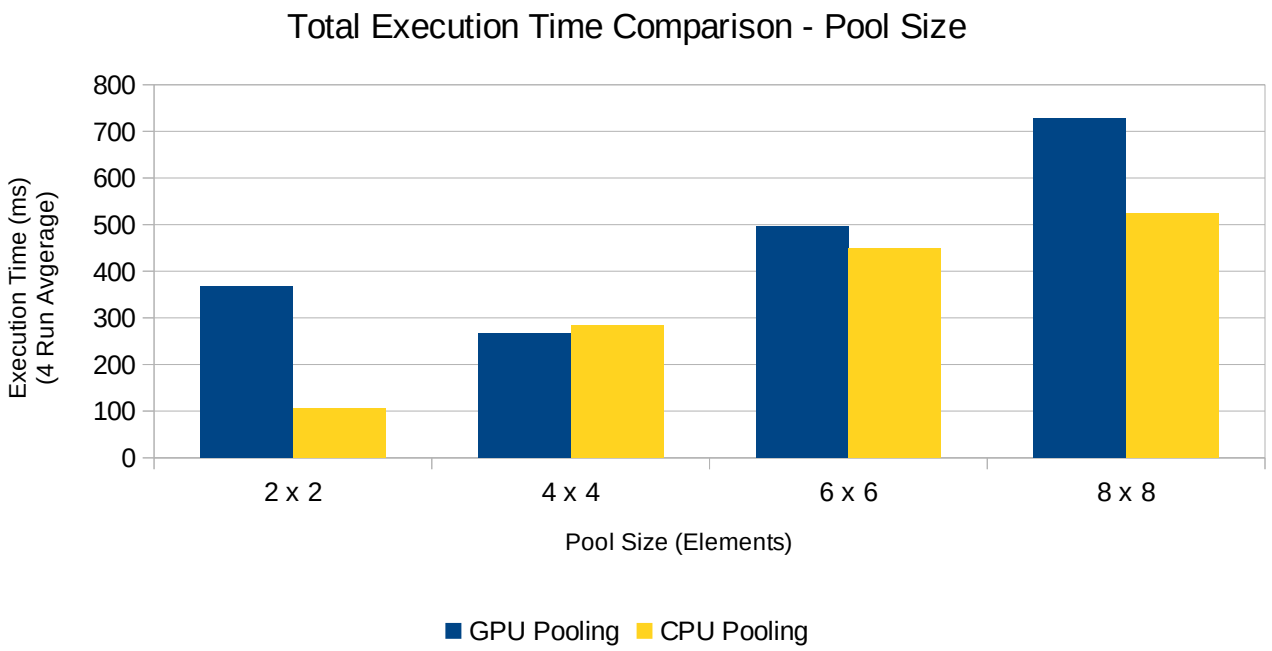


Figure 6: All execution times were measured using std:chrono at the call of the actual algorithm (i.e. poolLayer_gpu). Input size was kept constant at 32 x 32 elements, 3 channels, stride of 1 x 1. Note that the results are from a 4 run average.

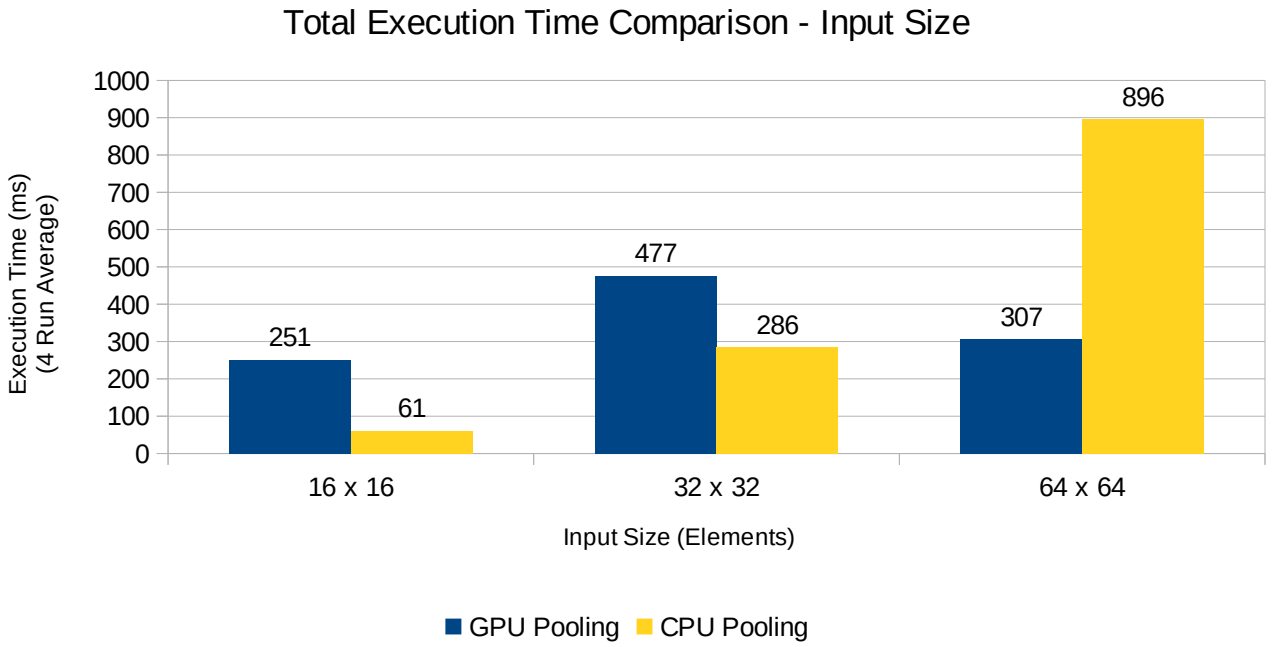


Figure 7: All execution times were measured using std:chrono at the call of the actual algorithm (i.e. poolLayer_gpu). Pool size was kept constant at 4 x 4 elements, stride of 1 x 1, 3 channels. Note that the results are from a 4 run average.

ECE60827 CUDA Assignment 2
Jerry Liu, 2/25/2023

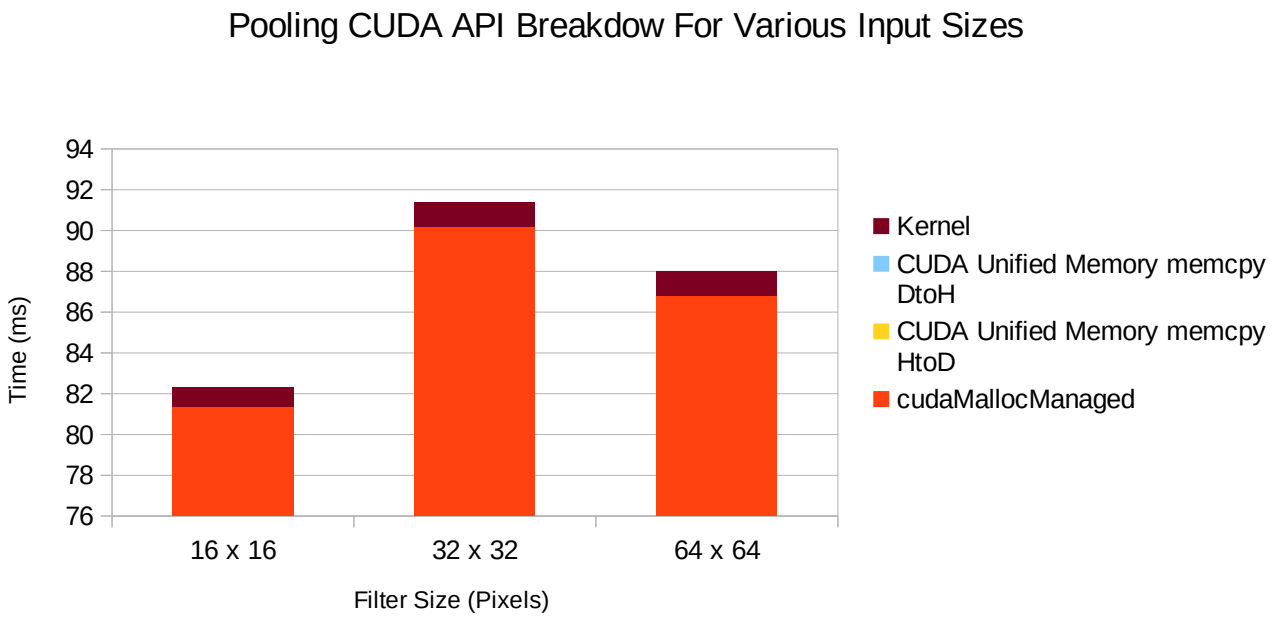


Figure 8: Breakdown of optimized median filter API calls using command "nsys nvprof ./lab2" Non-relevant API calls are excluded (i.e. cudaLibraryLoadData) for clarity. Pool size was kept constant at 4 x 4 elements, stride of 1 x 1, 3 channels. Note that the results are from a 4 run average.

Observations:

Though it might initially seem that the GPU implementation of pooling is slower in some cases, the reason behind the higher execution time appears to be run-to-run variance. The GPU implementation's total execution time would regularly jump between ~200ms to as much as ~1200ms depending on the inputs. This could be caused by many things, but the API breakdown shows that it is most likely caused by the variance in memory allocation and transfers. The kernel's execution time remains relatively constant across different input sizes. From this behavior, it can be assumed that this workload is more memory bound because the computational requirements are very low per output element when compared to median filtering. Analysis from nsight-compute (not shown) also supports this, with it reporting that the hardware occupancy is low. Ultimately, this variance would likely be negligible with larger input sizes, and the preliminary data from figure 7 supports this idea. It should be noted, however, that the GPU implementation of the pooling algorithm cannot handle very large problem sets because the system runs out of shared memory.